PSAI_088

CoinFabrik On Ink Integration Tests 2

- **Team Name:** CoinFabrik (Nektra S.A)
- **Payment Address:** 0xf488039EDe6B38D7689fDCC6A9FC2dd0EF39D54e (USDC)
- **Level:** 2

Project Overview :page_facing_up:

Overview

We have discovered that integration tests for ink! contracts lack some of the functionalities, or present implementation differences, when compared to E2E testing.

Integration tests run significantly faster than E2E (end-to-end) tests. If a full range of functionalities were provided, it could reduce testing and QA times.

Our intention is to `flatten the anvil` of ink! integration testing. With a properly flattened anvil, quality tools can be built.

Project Details

We have conducted a comprehensive analysis to identify any missing functionalities in integration tests and implementation differences with E2E tests, and to propose and develop new testing features based on our findings. This analysis was carried as part of a previous grant (link).

With this new grant, our objective is to implement our findings. Specifically, we aim to address functions in integration testing that have missing implementations or show differences when compared to e2e tests. We will add our contributions into the ink! project repository following existing contribution guidelines.

Ecosystem Fit

Having a comprehensive set of functionalities available for integration tests would bring numerous benefits to the entire community, including improved reliability, code quality and maturity, and faster feedback loops.

Integration tests are useful during their development and they are quicker than E2E tests. We learned this while working on fuzzing detection techniques during the Proof of Concept of Scout, which we performed in collaboration with researchers from the University of Buenos Aires. We believe that having a complete set of functionalities for integration tests would be useful for other teams working in the development of ink! smart contracts.

Team :busts_in_silhouette:

Team members

- Ariel Wassbein, Head of Research.
- Agustin Aon, Technical Lead.
- Valeria Caracciolo, Business Development.
- CoinFabrik’s development and QA teams.

Contact

- **Contact Name:** Valeria Caracciolo
- **Contact Email:** valeria.caracciolo@coinfabrik.com
- **Website:** https://www.coinfabrik.com/

Legal Structure

- **Registered Address:** Dr. Emilio Ravignani 2394, C1425 CABA, Argentina.
- **Registered Legal Entity:** Nektra S.A.

Team’s experience

We are a research and development company specialized in Web3, with a strong background in cybersecurity. Founded in 2014, we have worked on over 200 blockchain-related projects, EVM based and also for Solana, Algorand, and Polkadot. Beyond development, we offer security audits through a dedicated in-house team of senior cybersecurity professionals, currently working on code in Substrate, Solidity, Clarity, Rust, and TEAL.

Our team has an academic background in computer science and mathematics, with work experience focused on cybersecurity and software development, including academic publications, patents turned into products, and conference presentations. Furthermore, we have an ongoing collaboration on knowledge transfer and open-source projects with the University of Buenos Aires.

As well, CoinFabrik has been providing Quality Assurance as a service to development teams, accumulating valuable expertise in the field for a considerable period of time. Our clients highly appreciate this service, and as a result, we are eager to expand our capabilities to the ink! ecosystem.

Team Code Repos

- https://github.com/CoinFabrik
- https://github.com/CoinFabrik/on-ink-integration-tests
- https://github.com/CoinFabrik/scout
- https://github.com/CoinFabrik/web3-grant

Team LinkedIn Profiles (if available)

- https://www.linkedin.com/in/arielwaissbein/
- https://www.linkedin.com/in/agustin-aon/
- https://www.linkedin.com/in/valeriacaracciolo/

Development Status :open_book:

We have identified 24 functions exposed for their usage in integration and E2E tests in the file env_access.rs of the ink! repository. Of these 24 functions, we determined that there are 9 functions to work on with explicit plans, and 13 functions for which there might be implementation differences that remain to be analyzed. Two functions were deemed unfeasible for their implementation in the integration testing environment.

Table 1: Status of functions exposed in integration and e2e testing environments.

Issue Number	Function	Implemented Integration Tests	Implemented Integration E2E Tests	Status
1	default_accounts()	Yes	Yes	Implementation Difference.
2	set_contract_storage()	Yes	Yes	Missing limitation on Integration Testing.
3	invoke_contract_delegate()	No	Yes	Missing Function Implementation on Integration Testing.
4	invoke_contract()	No	Yes	Missing Function Implementation on Integration Testing
5	gas_left()	No	Yes	Missing Function Implementation on Integration Testing. Unfeasible Implementation.
6	set_code_hash()	No	Yes	Missing Function Implementation on Integration Testing.
7	instantiate_contract()	No	Yes	Missing Function Implementation on Integration Testing.
8	caller_is_origin()	No	Yes	Missing Function Implementation on Integration Testing.
9	code_hash()	No	Yes	Missing Function Implementation on Integration Testing.
10	own_code_hash()	No	Yes	Missing Function Implementation on Integration Testing.
11	call_runtime()	No	Yes	Missing Function Implementation on Integration Testing. Unfeasible Implementation.
12	caller()	Yes	Yes	Pending Analysis for Corrections in Implementation Differences.
13	transferred_value()	Yes	Yes	Pending Analysis for Corrections in Implementation Differences.
14	weight_to_fee()	Yes	Yes	Pending Analysis for Corrections in Implementation Differences.
15	block_timestamp()	Yes	Yes	Pending Analysis for Corrections in Implementation Differences.
16	account_id()	Yes	Yes	Pending Analysis for Corrections in Implementation Differences.
17	balance()	Yes	Yes	Pending Analysis for Corrections in Implementation Differences.
18	block_number()	Yes	Yes	Pending Analysis for Corrections in Implementation Differences.
19	minimum_balance()	Yes	Yes	Pending Analysis for Corrections in Implementation Differences.
20	terminate_contract()	Yes	Yes	Pending Analysis for Corrections in Implementation Differences.
21	transfer()	Yes	Yes	Pending Analysis for Corrections in Implementation Differences.
22	hash_bytes()	Yes	Yes	Pending Analysis for Corrections in Implementation Differences.
23	hash_encoded()	Yes	Yes	Pending Analysis for Corrections in Implementation Differences.
24	ecdsa_recover()	Yes	Yes	Pending Analysis for Corrections in Implementation Differences.

For these two sets of functions, with explicit implementation plans and pending analysis, the following work remains to be performed. - The implementation and correction of implementation differences of the 9 functions with explicit plans. These are the functions with issue numbers 1, 2, 3, 4, 6, 7, 8, 9, 10. - An analysis of the remaining 13 functions, which are implemented both for integration and E2E tests, in order to first estimate and then correct implementation differences (if any). These correspond to functions issue numbers 12 through 24. - QA: Adding tests to integrate the functions we add or modify to the ink! project repository following existing contribution guidelines. - Report Describing our Contribution.

Considering the dependency of several functions on the implementation of `instantiate_contract()`, we propose to split the work above into two milestones. All these implementations or modifications will be pushed into the ink! project repository following existing contribution guidelines.

We have also identified a bug in the e2e tests. When contracts are in a workspace with dependencies defined in `Cargo.toml`, and these dependencies are inherited in contracts, the e2e tests fail to compile. However, manually specifying dependencies in each contract resolves the issue. We've logged this bug on GitHub Issue #1919 and will be addressing it as part of our work in Milestone 1.

Development Roadmap :nut_and_bolt:

Overview

- **Total Estimated Duration:** 4 weeks
- **Full-Time Equivalent (FTE):** 4 FTE (0.50 Project Manager, 0.50 Tech Lead, 1 Full time Sr Rust Developer, 1 Full Time SemiSr Rust Developer, 1 Full Time QA Specialist)
- **Total Costs:** 30,000 USD

Milestone 1: Execution and Further Analysis

- **Estimated duration:** 4 weeks
- **FTE:** 4
- **Costs:** 30,000 USD

Number	Deliverable	Specification
0a.	License	MIT
		We will write a comprehensive report that compares the functionalities of integration tests and E2E (End-to-End) tests. This report will focus on the functions to be implemented/corrected in this milestone, corresponding to issues 1-default_accounts(), 2-set_contract_storage() and 7-

0b.	Documentation	instantiate_contract(). Documentation and test cases will be provided for the 13 functions with remaining analysis. If implementation differences are found in these functions, an estimate for their correction and an implementation idea will also be provided in our report. If applicable, we will suggest additional tests outside of the scope of this milestone. Particularly, for functions declared outside of the env_access.rs file, but that could be related to integration or e2e testing.
0c.	Testing and Testing Guide	The newly developed functionalities will be documented and tested following existing contribution guidelines. A testing guide will be included.
0d.	Docker	Does not apply at this stage.
0e.	Article	We will publish an updated report summary in our blog at https://blog.coinfabrik.com/.
1	Develop	We will develop the missing functionalities or correct implementation differences for functions 1-default_accounts(), 2-set_contract_storage() and 7-instantiate_contract(). All these implementations or modifications will be pushed into the ink! project repository following existing contribution guidelines. If applicable, we will develop additional tests or make ad hoc improvements to the ink codebase necessary for the completion of this milestone. Particularly for functions declared outside the env_access.rs file that might be related to integration or end-to-end testing.
2	Review and Estimate	We will review the remaining 13 unanalysed functions, which are implemented both for integration and e2e tests. For these functions we will provide documentation, a test case and an implementation estimation if applicable. These correspond to functions issue numbers 12 through 24.
3	Quality Assurance	We will adhere to existing contribution guidelines and add necessary tests to integrate the new implemented or corrected functions to the ink! project repository.

Future Plans

After finishing the Milestone 1: Execution and Further Analysis, we will submit a new grant proposal to continue with the implementation of the remaining functions. We will include specific references to developments associated with the estimations resulting from the further analysis of functions issue numbers 12 through 24.

Next Milestone: Execution

- **Estimated duration:** 4 weeks
- **FTE:** 4
- **Costs:** 30,000 USD

Number	Deliverable	Specification
0a.	License	MIT
0b.	Documentation	We will write a comprehensive report that compares the functionalities of integration tests and E2E (End-to-End) tests. This report will focus on the the functions to be implemented in this milestone, corresponding to issues 3-invoke_contract_delegate(), 4-invoke_contract(), 6-set_code_hash(), 8-caller_is_origin(), 9-code_hash(), 10-own_code_hash(). Our report will also document the implementation of any missing functionalities, or correct implementation differences, for the 13 functions with issues 12 through 24. For this group, we will document any additional work that was required in order to ensure consistency between integration and e2e tests. If applicable, we will suggest additional tests outside of the scope of this milestone. Particularly, for functions declared outside of the env_access.rs file, but that could be related to integration or e2e testing.
0c.	Testing and Testing Guide	The newly developed functionalities will be documented and tested following existing contribution guidelines. A testing guide will be included.
0d.	Docker	Does not apply at this stage.
0e.	Article	We will publish an updated report summary in our blog at https://blog.coinfabrik.com/.
		We will implement the missing functionalities or resolve implementation differences for function issues 3-invoke_contract_delegate(), 4-invoke_contract(), 6-

1	Development	set_code_hash(), 8-caller_is_origin(), 9-code_hash(), 10-own_code_hash(). We will implement any missing functionalities, or correct implementation differences, for the 13 functions with issues 12 through 24. For this group, we will document any additional work required in order to ensure consistency between integration and e2e tests. All these implementations or modifications will be pushed into the ink! project repository following existing contribution guidelines. If applicable, we will develop additional tests or make ad hoc improvements to the ink codebase necessary for the completion of this milestone. Particularly for functions declared outside the env_access.rs file that might be related to integration or end-to-end testing.
2	Quality Assurance	We will adhere to existing contribution guidelines and add necessary tests to integrate the new implemented or corrected functions to the ink! project repository.

Moving forward, we have two projects in mind:

- Research and develop an advanced testing automation solution for ink! smart contracts.
- Improve our open source bug-detection tool Scout.

Referral Program (optional) :moneybag:

Additional Information :heavy_plus_sign:

How did you hear about the Grants Program? Richard Casey from Parity brought this program to our attention, and we have already successfully delivered two applications as a result.

During our inquiries for this application, we briefly consulted Sam Ruberti from the ink! Team and David Hawig from the Web3 Foundation. Their encouragement motivated us to proceed with this presentation.